



Análisis Léxico

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Introducción	1
2. Alfabeto	2
3. Expresiones Regulares	3
3.1. Algoritmo de Flex	4
3.2. Token	5
4. Contextos	6
5. Detalles de Implementación	7
5.1. Interfaz Analizador Léxico-Sintáctico	7
5.2. Estado del Analizador Léxico	8
5.3. Acciones	9
6. Bibliografía	9

1. Introducción

Sea $G = \langle \Sigma, N, \Pi, S \rangle$ una gramática, donde Σ es el alfabeto, N es el conjunto de símbolos no-terminales, Π es el conjunto de producciones y S es el símbolo no-terminal inicial, se define un **analizador léxico** como aquel cuyo objetivo principal es transformar una secuencia de símbolos en una nueva secuencia compuesta exclusivamente por símbolos del alfabeto Σ .

Para las siguientes secciones, el analizador léxico utilizado será Flex, pero las ideas y conceptos son generales a otros analizadores.

2. Alfabeto

Luego de diseñar la sintaxis de un nuevo lenguaje formal (o de disponer de la especificación de uno existente), debe ser posible tomar un programa de entrada como el siguiente:

```
[HttpGet]
[ProducesResponseType(200)]
public async Task<ActionResult<Manifest>> GetManifestAsync() {
    return new Manifest();
}
```

y construir una secuencia equivalente de símbolos pero que se restrinja a aquellos símbolos de un alfabeto Σ definido para la gramática del lenguaje en cuestión (en el ejemplo, C#).

Los motivos por los que se debe realizar un mapeo entre esta secuencia de símbolos (idealmente codificados en UTF-8 según el estándar Unicode), y los símbolos de Σ , son al menos dos:

- (I). **Finitud:** Naturalmente un alfabeto es finito por definición, sin embargo, tomando casi cualquier lenguaje como ejemplo, es claro que uno podría definir un símbolo atómico por cada identificador de variable o función, o por cada número entero, con lo cual el alfabeto debería ser infinito. Para eliminar esta situación, un conjunto infinito de símbolos en el programa de entrada se mapean hacia un único símbolo atómico en Σ .
- (II). **Rendimiento:** En general, un analizador léxico es menos potente que un analizador sintáctico (la siguiente fase del compilador), en términos de su capacidad de computabilidad (i.e., lo que puede o no puede hacer, independientemente de su capacidad de procesamiento y almacenamiento). Sin embargo, para mejorar la eficiencia se emplea un autómata que permite realizar el mapeo de símbolos (y el agrupamiento de los mismos), utilizando recursos más limitados, incrementando el ancho de banda y reduciendo la carga en el analizador sintáctico debido a que la cantidad de símbolos es inferior (debido al agrupamiento de los mismos).

Por ejemplo, es factible realizar los siguientes mapeos desde secuencias de símbolos codificadas en UTF-8 hacia un único símbolo de Σ :

"Al principio Dios creó el cielo y la tierra."	
"La tierra era algo informe y vacío,"	
"las tinieblas cubrían el abismo,"	→ STRING
"y el soplo de Dios se cernía sobre las aguas."	
0	
123	→ INTEGER
10000	

ActionResult	
x	
y	→ ID
User	
Credential	

Evidentemente, las secuencias de símbolos a la izquierda representan conjuntos infinitos (fragmentos de texto, números enteros e identificadores de variables o funciones, respectivamente), pero son fácilmente identificables y representados por símbolos atómicos finitos, suponiendo que $\Sigma = \{ ID, INTEGER, STRING \}$, en este caso.

Adicionalmente, está claro que la transformación de las secuencias de entrada incrementa el ancho de banda hacia la segunda fase, por ejemplo, los 4 extractos del libro del Génesis utilizan 156 símbolos (160 bytes codificados en UTF-8), pero luego del mapeo solo 4 (uno por cada texto). Si este símbolo STRING de Σ se codifica mediante un entero de 2 bytes, entonces se utilizarían 8 bytes en lugar de 160 (20 veces mejor).

Entonces, ¿qué secuencias forman parte del alfabeto? Todos aquellos símbolos o agrupamientos de símbolos que tengan valor o semántica (significado) por sí mismos y que carezcan del mismo si se aplica una subdivisión, indicando que efectivamente son atómicos.

Ejemplos claros de símbolos del alfabeto son: palabras reservadas, conjuntos literales de ciertos tipos de datos (enteros, reales, cadenas de texto, caracteres, identificadores de variables o funciones, etc.), símbolos de operadores aritméticos, relacionales, lógicos o matemáticos en general, y signos de puntuación o agrupamiento (paréntesis, llaves, comillas, puntos y comas, dos puntos, etc.).

3. Expresiones Regulares

⚠ IMPORTANTE: Las acciones de Flex utilizadas en esta sección se corresponden al modo de operación PULL. La diferencia está en que las mismas devuelven la etiqueta del token (*token label*), mientras que en el nuevo modo PUSH, deben devolver el estado de la compilación (la etiqueta viaja por otro medio). Este nuevo mecanismo se introdujo en la versión **v2.0.0**, es decir, las versiones anteriores utilizan el modo PULL.

El mapeo de un símbolo o grupo de símbolos en la entrada del analizador léxico hacia un símbolo del alfabeto Σ , se realiza en general mediante el uso de *autómatas finitos determinísticos* (DFA), debido a su simplicidad pero a la vez eficiencia y composicionalidad (i.e., la capacidad de unir múltiples autómatas para construir uno más complejo).

Debido a que un DFA es equivalente a una expresión regular, los analizadores léxicos usualmente se construyen automáticamente desde una lista de expresiones regulares que el programador asocia con cada símbolo del alfabeto final. De esta manera, las expresiones regulares permiten definir tanto conjuntos finitos o infinitos de posibles secuencias de caracteres que serán mapeadas eficientemente a sus correspondientes símbolos de Σ .

En particular, Flex es un generador de analizadores léxicos que opera de esta manera: toma una especificación de expresiones regulares (desde un documento *.l), junto con sus mapeos hacia Σ , y emite un analizador léxico eficiente (también llamado *escáner*), basado en un DFA.

Utilizando como base el proyecto base [Flex-Bison-Compiler](#), se pueden identificar las siguientes reglas de mapeo:

```
"_"           { return SUB; }
"*"           { return MUL; }
"/"           { return DIV; }
"+"           { return ADD; }

"("           { return OPEN_PARENTHESIS; }
")"           { return CLOSE_PARENTHESIS; }

[[:digit:]]+   { return INTEGER; }

[[:space:]]+   { ; }
.              { return UNKNOWN; }
```

Nótese que al igual que en los ejemplos anteriores, del lado izquierdo se encuentra la expresión regular que acepta las secuencias válidas de entrada, mientras que el lado derecho de esta especificación efectivamente emite un símbolo del alfabeto $\Sigma = \{ SUB, MUL, DIV, ADD, (...) \}$.

Cabe destacar algunas situaciones particulares. Por ejemplo, en el caso de la secuencia definida por la expresión regular `[[:space:]]+`, no se emite **ningún mapeo**, ya que esta secuencia representa grupos de espacios en blanco (*whitespace*), lo que produce un DFA que ignora dichas secuencias, y por lo tanto la siguiente fase de análisis sintáctico evitará sus efectos en la gramática de forma transparente.

Por otro lado, la expresión regular final emite un símbolo UNKNOWN, utilizado para representar un concepto que no es propio de Σ , ya que representa una secuencia de símbolos desconocida, que no es posible mapear ni ignorar y que por lo tanto representa un **error de sintaxis**.

3.1. Algoritmo de Flex

¿En qué orden se evalúan los mapeos?

Flex utiliza el algoritmo **Longest Prefix Match**, lo que implica que siempre se intenta consumir la mayor cantidad de símbolos antes de realizar el mapeo y emitir un símbolo del alfabeto Σ . Por ejemplo, para los siguientes mapeos:

```
"class"       { return CLASS; }

[[:alpha:]][[:alnum:]]+ { return ID; }
```

y si la secuencia de entrada es "class123", se aplicará el segundo mapeo emitiendo un símbolo ID, ya que dicha regla permite consumir más símbolos que la primera, con lo cual, "class123" se interpretará correctamente como un identificador de variable en lugar de la keyword **class**.

Por otro lado, si la secuencia fuese "class 123", Flex retorna CLASS, ya que el espacio entre "class" y "123" impide que se ejecute la segunda regla.

En el caso de que múltiples expresiones regulares coincidan con la secuencia de entrada, por ejemplo sobre la cadena "unaVariable":

```
[[:alpha:]]*           { return TEXT; }  
[[:alpha:]][[:alnum:]]+ { return ID; }
```

tiene mayor precedencia quién aparece **primero**, en este caso, la regla de mapeo que emite el símbolo TEXT del alfabeto (nótese que en este caso, ambos mapeos coinciden con la cadena de entrada).

3.2. Token

El objetivo de un analizador léxico es transformar la secuencia de entrada en un **stream** o flujo de unidades atómicas denominadas **tokens**. Un token es la unión de un **atributo** y un **lexema**, donde lexema es literalmente la porción de símbolos de la cadena de entrada que se está mapeando hacia un símbolo del alfabeto, mientras que los atributos simplemente son metadatos o **valores semánticos** que se asocian al lexema para transportar información adicional hacia la siguiente etapa (el *analizador sintáctico*).

Por ejemplo, retomando el siguiente fragmento de un programa en C#:

```
[HttpGet]  
[ProducesResponseType(200)]  
public async Task<ActionResult<Manifest>> GetManifestAsync() {  
    return new Manifest();  
}
```

se puede descomponer en un flujo de lexemas (ignorando espacios):

```
"[", "HttpGet", "]", "[", "ProducesResponseType", "(", "200", ")", "]",  
"public", "async", "Task", "<", "ActionResult", "<", "Manifest", ">", ">",  
"GetManifestAsync", "(", ")", "{", "return", "new", "Manifest", "(", ")", ":",  
"}"
```

para luego mapearse hacia símbolos del alfabeto:

```
OPEN_BRACKET, ID, CLOSE_BRACKET, OPEN_BRACKET, ID, OPEN_PARENTHESIS, INTEGER,  
CLOSE_PARENTHESIS, CLOSE_BRACKET, PUBLIC, ASYNC, ID, LESS_THAN, ID, LESS_THAN,  
ID, GREATER_THAN, GREATER_THAN, ID, OPEN_PARENTHESIS, CLOSE_PARENTHESIS,  
OPEN_BRACE, RETURN, NEW, ID, OPEN_PARENTHESIS, CLOSE_PARENTHESIS, SEMICOLON,  
CLOSE_BRACE
```

Ahora bien, cada uno de estos símbolos puede o no acarrear metadatos (un atributo adicional), por ejemplo, podría ser relevante que el símbolo INTEGER contenga efectivamente el valor numérico 200 en su interior, y que los símbolos ID transporten el lexema original que los produjo, a modo de identificar esas variables, constantes o nombres de clases y funciones específicamente.

En este sentido, y representando un token como una tupla de tipo (Σ, V) , donde Σ es el alfabeto y V es el conjunto de valores semánticos que un símbolo de Σ puede tomar, se obtiene el flujo final de tokens:

```
(OPEN_BRACKET, ∅), (ID, "HttpGet"), (CLOSE_BRACKET, ∅), (OPEN_BRACKET, ∅), (ID,
"ProducesResponseType"), (OPEN_PARENTHESIS, ∅), (INTEGER, 200),
(CLOSE_PARENTHESIS, ∅), (CLOSE_BRACKET, ∅), (PUBLIC, ∅), (ASYNC, ∅), (ID,
"Task"), (LESS_THAN, ∅), (ID, "ActionResult"), (LESS_THAN, ∅), (ID, "Manifest"),
(GREATER_THAN, ∅), (GREATER_THAN, ∅), (ID, "GetManifestAsync"),
(OPEN_PARENTHESIS, ∅), (CLOSE_PARENTHESIS, ∅), (OPEN_BRACE, ∅), (RETURN, ∅),
(NEW, ∅), (ID, "Manifest"), (OPEN_PARENTHESIS, ∅), (CLOSE_PARENTHESIS, ∅),
(SEMICOLON, ∅), (CLOSE_BRACE, ∅)
```

Nótese que para los valores semánticos vacíos (donde no tiene sentido transportar un atributo o metadato más allá del símbolo del alfabeto), se utilizó $\emptyset$.

Transformada la secuencia de entrada en este flujo de tokens, el analizador sintáctico se puede encargar libremente de construir el árbol de sintaxis abstracta (AST).

4. Contextos

Si bien es una prestación que no pertenece a las máquinas de tipo DFA, Flex permite manipular el analizador léxico generado de una manera que le provee mayor potencia para separar la lista de mapeos en diferentes *grupos* o *contextos*.

Si una expresión regular permite mapear un lexema hacia un símbolo del alfabeto, entonces es factible pensar que bajo ciertas circunstancias, un mismo lexema se puede **reinterpretar** como diferentes símbolos.

Por ejemplo, una palabra compuesta de símbolos del alfabeto español podría considerarse el nombre de un identificador, y por lo tanto mapearse hacia un símbolo ID. Sin embargo, si la palabra se encuentra dentro de un comentario en el programa de entrada, debe ignorarse, o en casos más complejos (como aquellos donde se infieren datos desde documentación estructurada como Javadoc, JSDoc o Sphinx), debe interpretarse de otro modo.

Para manipular un lexema de múltiples maneras, se debe indicar el momento en el cual ocurre un **cambio de contexto**:

```
"/*"                               { BEGIN(MULTILINE_COMMENT); }
<MULTILINE_COMMENT>"*/"          { BEGIN(INITIAL); }
<MULTILINE_COMMENT>[[:space:]]+   { ; }
<MULTILINE_COMMENT>[^*]+         { ; }
<MULTILINE_COMMENT>.             { ; }
```

De esta manera, al identificar el lexema “/*” en la secuencia de entrada se ingresará al contexto denominado `MULTILINE_COMMENT`. Dentro de este contexto, el grupo de expresiones regulares **activas** queda comprendido por todas aquellas que comienzan con la etiqueta `<MULTILINE_COMMENT>`.

Esto tiene el efecto de apagar o encender cada grupo de expresiones regulares según sea conveniente, luego de identificar cierto patrón en la entrada. Es análogo a utilizar un DFA, pero cambiar a uno completamente diferente si este se topa con un lexema determinado en la entrada.

¿Cómo se sale del contexto? Ingresando al contexto por defecto de Flex: `INITIAL`. Este contexto ya viene predefinido y se utiliza para volver al DFA original. En este caso, se debe salir del contexto al identificar la secuencia “*/”, pero nótese que esta secuencia solo se identifica dentro del contexto `MULTILINE_COMMENT`, con lo cual si se halla fuera de él, Flex emitirá un error de sintaxis como es debido.

¿Cómo se crea un contexto? Para ello existe una directiva especial de Flex:

```
%x MULTILINE_COMMENT
```

En este caso el contexto `MULTILINE_COMMENT` es de tipo exclusivo, lo que implica que cuando se ingrese a dicho contexto, solo aquellas expresiones regulares etiquetadas con `<MULTILINE_COMMENT>` se encontrarán activas. Por otro lado, es posible utilizar `%s` en lugar de `%x` para definir un contexto inclusivo que, al contrario del exclusivo, permite que dentro del contexto también permanezcan activas aquellas reglas sin etiquetar (asociadas al contexto por defecto `INITIAL`).

5. Detalles de Implementación

Debido a que la implementación de Flex es una reescritura del viejo generador Lex, y que por decisión de diseño los autores optaron por mantener su interfaz de programación, la DX de la herramienta es significativamente pobre, con lo cual en el repositorio de base [Flex-Bison-Compiler](#), se desarrollaron ciertas abstracciones alrededor del analizador para simplificar o minimizar tanto la complejidad de la implementación como la posible introducción de errores de programación y *memory-leaks*.

5.1. Interfaz Analizador Léxico-Sintáctico

El analizador léxico se encarga de generar una secuencia o flujo de tokens (*token stream*). Esta secuencia será consumida por el analizador sintáctico o *parser* (implementado mediante la herramienta Bison), quien a su vez intentará construir un AST (Árbol de Sintaxis Abstracta), sobre dicha secuencia, a modo de representar la estructura sintáctica del programa de entrada.

Si bien el mecanismo de comunicación entre estas dos fases de análisis es la secuencia o flujo de tokens, existen al menos dos modos de control disponibles: **PULL** y **PUSH**.

El modo **PULL** implica que el analizador sintáctico solicita el siguiente token al analizador léxico directamente, cada vez que lo requiera. De este modo, un patrón en la entrada identificado

mediante el analizador léxico produce un token que el parser consume. El parser ejecuta todas las acciones semánticas derivadas de dicho consumo hasta detenerse. Luego repite el proceso; solicita un nuevo token del escáner o *lexer* hasta consumir el flujo completo. La limitación es clara: el analizador léxico solo puede emitir un token a la vez.

En el modo **PUSH**, en cambio, es el analizador léxico el que emite los tokens hacia el *parser*, con lo cual la limitación anterior desaparece. Un escáner puede emitir 0 (cero), 1 (uno), o múltiples tokens cada vez, aunque el patrón identificado en la entrada (el *lexema*), sea único.

La versión **v2.0.0** emplea el nuevo modo PUSH, que es particularmente útil para resolver la sintaxis de lenguajes que utilizan indentación basada en espacios, similar a Python o Haskell (ver Fromme, 2007).

Dado que el modo afecta la comunicación entre Flex y Bison, pero no la semántica general (es decir, se sigue transfiriendo una secuencia de tokens al igual que bajo el modo PULL), el *parser*, particularmente sus acciones semánticas, no sufren modificación alguna.

5.2. Estado del Analizador Léxico

[MODO PULL] La abstracción principal consiste en agrupar el estado del analizador léxico dentro de una instancia de la clase [LexicalAnalyzerContext](#), la cual se puede inyectar en cada acción de Flex mediante la función `createLexicalAnalyzerContext()`, quien se encarga de crear una instancia completamente nueva en cada llamado utilizando memoria dinámica.

Flex se basa en el uso intensivo de variables globales a lo largo de su implementación (`yytext`, `yytext`, `yytext`, etc.), y debido a que esto es una mala práctica, se optó por copiar el contenido de dichas variables durante la instanciación de un nuevo contexto (no confundir este contexto, con el sistema de contextos utilizado para los comentarios multilínea presentado en la [Sección § 4: Contextos](#)).

Actualmente el contexto inyectado provee acceso al siguiente estado:

- (I). **currentContext**: Un número que indica el número de contexto actual (`INITIAL`, `MULTILINE_COMMENT`, etc.), el cual es especialmente útil si el lenguaje requiere del uso de contextos anidados. Por defecto, el contexto `INITIAL` siempre se representa mediante el número 0 (cero).
- (II). **length**: El largo en caracteres del lexema que se activó (según la semántica de la función estándar `strlen`).
- (III). **lexeme**: El lexema propiamente dicho, copiado y reservado mediante memoria dinámica, lo que permite modificar el mismo libremente sin destruir el estado interno de Flex.
- (IV). **line**: El número de línea dónde Flex encontró el lexema activo.
- (V). **semanticValue**: El atributo asociado al lexema que conformará el token final. La definición de este campo se detalla en [📖 \(2024-05-08, v0.1.0\) Análisis Sintáctico](#), ya que sus detalles son provistos por el generador Bison.

5.3. Acciones

[MODO PULL] Las acciones de Flex se colocan a la derecha de cada expresión regular tal cual se mostró en la **Sección § 3: Expresiones Regulares**. Estas acciones admiten código en C de cualquier tipo, pero lo usual es que se respeta el siguiente esquema idiomático¹ para mantener tanto la consistencia como la legibilidad del código:

- (I). Una función dedicada a cada expresión regular, como por ejemplo es el caso de la acción `IntegerLexemeAction()`, aunque también es válido agrupar múltiples expresiones bajo la misma función, si es posible determinar el token a emitir desde el lexema, como ocurre con `ArithmeticOperatorLexemeAction()` que manipula todos los operadores. La función deberá retornar el token asociado al lexema, `UNKNOWN` en caso de que el lexema sea inválido, o nada (`void`), si el lexema debe ser ignorado por completo.
- (II). La inyección sobre cada acción de un nuevo contexto mediante el llamado a `createLexicalAnalyzerContext()`.
- (III). El uso de la directiva `BEGIN` al inicio de la acción al ingresar a un contexto, y al final de la misma sobre la acción de salida del mismo (esto garantiza que las acciones se ejecutan dentro del contexto por completo).

6. Bibliografía

- (I). Estes, W. (2024) *Lexical Analysis With Flex, for Flex v2.6.2* (accedido el 2024-09-25).
Sección § 6: Patterns
Sección § 7: How the Input Is Matched
Sección § 10: Start Conditions
Sección § 11: Multiple Input Buffers
- (II). Fromme, O. (2007) [Python: Myths about Indentation](#) (accedido el 2025-09-03).
- (III). Levine, J. R. (2009) [flex & bison](#). O'Reilly Media, Inc.

¹ Entiéndase por *idiomático* a una forma de ordenar o implementar ciertas construcciones dentro de la aplicación en el lenguaje subyacente (C, en este caso).